Distributed In-Memory Cache with Strong Consistency Guarantees

(Distributed In-Memory Cache
with Strong Consistency Guarantees)

Dominik Szczepaniak

Praca inżynierska

Promotor: dr Piotr Witkowski

Uniwersytet Wrocławski
Wydział Matematyki i Informatyki
Instytut Informatyki

21 listopada 2025

Abstract

. . .

. . .

Contents

Chapter 1

Introduction

1.1 What am i building - cache, what is it purpose

1.1.1 Use cases of such software

The distributed cache with strong consistency would be perfect solution for any application that doesn't work in centralized architecture and needs to ensure that the data it returns is always correct and can't afford to overload underlying databases for frequent operations. Let's look at some examples.

- Any financial transaction broker as if the data is incorrect the bank, or anyone making a transaction, might be at loss
- Inventory management systems, such as online shops needs data quickly and the state of the data needs to be consistent
- Games might benefit from this application aswell, for example the leaderboard might change very frequently but still need accurate data for every player on the server
- Session management on web servers for many cases, for example authentication
- Industrial software might need to check the system health based on some services data, this data cannot be wrong or contradictory to eachother

1.2 What is consensus

1.3 What type of applications do need a consensus at all times

Chapter 2

Reaching consensus

2.1 CAP theorem (ADD CITATION)

CAP theorem states, that the system cannot simultaneously satisfy all three following guarantees:

- Availability - Every request receives a response, without a guarantee that it contains the most recent version of the information. The system remains operational even if some nodes fail.
- Consistency - Every read receives the most recent write or an error. This is a single, up-to-date copy of the data.
- Partition tolerance - The system continues to operate even if there are network failures that cause a "partition" where some nodes cannot communicate with others.

2.1.1 Which part of the CAP theorem do I fulfill

Raft algorithm will always focus on consistency as it's main goal, hence this part of the system is always guaranteed. Let's look at the remaining guarantees - availability and partition tolerance.
In the worst case scenario where the network partitions and our node splits into some number of subsets of servers that cannot connect to each other in any way the Raft algorithm will not be able to elect a leader if the size of servers subset is smaller than majority. If it cannot elect a leader, then it cannot proceed with any operation. In this case the entire system is unavailable until it "merges" back into the size of the majority. That case shows that the system does not fulfill availability from CAP theorem. It fulfills the partition tolerance, because each subset of servers will continue to operate normally.

2.2 What type of algorithms are these - byzantine vs non byzantine

2.2.1 Why do we not want a byzantine algorithm

2.3 How can we reach consensus - about algorithms

Chapter 3

Raft

3.1 What is Raft

3.2 How does Raft work

Chapter 4

System architecture

The design of the system's routing, partitioning, and consistency logic evolved through distinct iterations to balance the conflicting requirements of sub-millisecond latency (Cache) and strong data safety (Consistency). The final architecture leverages a Hybrid Control/Data Plane model, using Raft for topology management and Synchronous Replication for data propagation.

4.0.1 Initial Concept: The Gateway Model

The initial design proposed a classic "Proxy" architecture to abstract the cluster topology from the client.

Design:

- A centralized **Gateway Service** acts as the entry point for all client requests.
- The Gateway maintains the list of active storage nodes (N).
- Routing is determined by Dynamic Modular Hashing: target=hash(k)(modN).

Critique & Rejection: While simple, this model was rejected due to two critical flaws:

1. **The "Double-Hop" Latency Penalty:** Every operation requires two network traversals (Client → Gateway → Node). For an in-memory cache, this overhead is prohibitive.
2. **The "Thundering Herd" Problem:** The formula is tightly coupled to node count (N). If N changes, roughly 75% of keys are remapped, rendering the cache unusable during scaling.

4.0.2 Structural Optimization: Fixed Partitioning

To solve the "Thundering Herd" problem, the system adopts the Fixed Partition model. Keys are mapped to logical buckets rather than physical nodes. PartitionID=CRC16(key)(mod1024)

The constant 1024 ensures that adding a node results in minimal, deterministic data movement.

4.0.3 The Consistency Model: Defining "Strong Cache Consistency"

A core thesis requirement is "Strong Consistency." To achieve this without becoming a slow, disk-bound database, the architecture makes a precise distinction between Consistency and Durability.

- **Durability (Database Domain):** Guaranteeing data survives a total power failure (requires disk fsync). This is *out of scope* for this in-memory cache.
- **Consistency (Thesis Scope):** Guaranteeing that once a write is acknowledged, it is visible to all subsequent reads and survives individual node failures.

To achieve this, the system rejects "Async Replication" (Redis-style, where data loss is possible) and "Full Consensus" (CockroachDB-style, where latency is too high). Instead, it implements Synchronous Primary-Backup Replication.

4.0.4 Final Architecture: The Supervisor-Worker Model

The system is bifurcated into two distinct layers:

1. The Control Plane ("The Supervisor")

This layer consists of a small cluster (typically 3 nodes) running the Raft Consensus Algorithm.

- **Responsibility:** It manages the **Partition Table** (The Map).
- **State:** It stores the mapping: Partition P→Primary: Worker A, Backup: Worker B.
- **Role:** It acts as the "Judge." It does not store cache data. It only prevents Split-Brain scenarios by enforcing that only one valid Primary exists for any partition at any time.

2. The Data Plane ("The Workers")

This layer consists of N worker nodes that store the actual Key-Value pairs in memory. Crucially, **Worker Nodes do not run Raft**.

- **Responsibility:** High-throughput memory storage and replication.
- **Replication Factor:** The system uses N=2 (1 Primary + 1 Backup). This creates a trade-off where the system is resilient to single-node failures while minimizing network latency (compared to N=3 or higher).

3. The Write Flow (Synchronous Replication)

To guarantee Strong Consistency, the Client communicates directly with Workers using a "Smart Client" approach, enforcing a strict replication chain.

The Execution Chain:

1. **Lookup:** The Client checks its cached Partition Table to find the Primary Worker for the key.
2. **Direct Send:** The Client sends `SET key=val` to the **Primary**.
3. **Lock & Forward:** The Primary locks the key locally and immediately sends a `REPLICATE` command to the assigned **Backup Worker**.
4. **Wait for ACK:** The Primary **pauses** execution. It does not reply to the client.
5. **Backup Confirm:** The Backup writes to memory and sends an `ACK` to the Primary.
6. **Completion:** Only upon receiving the Backup's `ACK` does the Primary return `OK` to the Client.

Failure Analysis: If the Primary crashes *before* sending the final `OK`, the client treats the request as failed. If the Primary crashes *after* sending `OK`, the data is guaranteed to be on the Backup. The Raft Supervisor then detects the failure and promotes the Backup to Primary, preserving the data and maintaining Strong Consistency.

4.1 Entry point

The entry point defines how external applications interact with the distributed cache. To ensure the system meets the requirements of low latency and high availability, I analyzed two distinct approaches: the Reverse Proxy model and the Smart Client model.

4.1.1 Rejected Approach: The Reverse Proxy

In a traditional architecture, a load balancer or reverse proxy sits between the client and the cluster.

- **Mechanism:** The client sends a request to a generic address (e.g., cache.api). The proxy looks up the internal topology and forwards the request to the correct node.
- **Flaw (The Consensus Paradox):** You noted a critical flaw: *"Do proxies need consensus?"* If proxies cache the topology state to be fast, they risk having stale routes (routing to a dead node). If they don't cache state, they become a performance bottleneck. Furthermore, the proxy adds an extra network hop (Client $\rightarrow$ Proxy $\rightarrow$ Node), effectively doubling the network latency for every operation.

For these reasons, the Proxy model was rejected.

4.1.2 Selected Approach: The Smart Client

The system implements a "Smart Client" library (Driver) that is embedded directly into the user's application.

Responsibility: The Smart Client is not merely a TCP connector; it is a fully topology-aware participant in the protocol.

1. **Topology Caching:** On startup, the client connects to *any* active Worker or Supervisor and downloads the **Partition Table**.
2. **Local Routing Decision:** For a command SET key value, the client performs the partitioning logic locally:

$$target_node = PartitionTable[CRC16(key) (mod 1024)].Primary$$

3. **Direct Connection:** The client opens a persistent TCP connection directly to the target Worker Node.

Failure Correction: If the client's Partition Table is stale (e.g., a node moved recently), the Worker will reject the request with a WRONG_NODE error. The client catches this error, queries the Supervisor for a new Partition Table, and retries the operation. This guarantees that after one network round-trip, the client's view of the cluster is eventually consistent with the Raft state.

4.2 Component 2: Data Partitioning Strategy

4.2.1 The Algorithm: Fixed Slot Partitioning

To achieve predictable scaling, the system eschews dynamic modular hashing (hash(modN)) in favor of a Pre-Sharded model.

The key-space is divided into a static number of logical units called Partitions (or Slots).

- **Total Partitions (P):** Fixed at 1024.
- **Mapping Function:** PartitionID=CRC16(key)(mod1024).

4.2.2 Assignment Logic

Physical nodes do not own keys directly; they own Partitions.

- **Worker A** might own Partitions [0,340].
- **Worker B** might own Partitions [341,680].

When a new node (Worker C) joins the cluster, the system does not re-hash keys. Instead, the Supervisor simply reassigns ownership of specific partitions (e.g., taking [0,50] from A and [341,390] from B) to Worker C. This minimizes data movement and eliminates the "Thundering Herd" problem.

4.3 Component 3: The Control Plane (The Supervisor)

(Note: This replaces your "Node" section. "Supervisor" is more precise than "Node" for the Raft part.)

The Supervisor is a dedicated server (or set of 3 servers) responsible for the cluster's metadata. It acts as the "Brain" of the system.

4.3.1 The Raft Consensus Role

The Supervisor runs the Raft Consensus Algorithm. Its Finite State Machine (FSM) manages exactly one data structure: the Partition Table. State=Map⟨PartitionID→PrimaryIP,Backup

4.3.2 Responsibilities

1. **Membership Management:** Tracks which Worker nodes are alive via heartbeat monitoring.

2. **Topology Enforcement:** If a Worker dies, the Supervisor detects the failure, updates the Partition Table (promoting a Backup to Primary), and commits this change via Raft.

3. **Source of Truth:** It serves the current Partition Table to Clients and Workers upon request.

Crucially, the Supervisor never stores or processes cache data (GET/SET). This isolation ensures that heavy cache traffic cannot destabilize the consensus mechanism.

4.4 Component 4: The Data Plane (The Worker)

(Note: This replaces your "Server" and "Application" sections. This is the core implementation.)

The Worker is a high-throughput server responsible for storing data and handling client requests. Workers do **not** run the Raft algorithm.

4.4.1 Storage Engine

Each Worker maintains an in-memory hash map (e.g., Go Map or C++ Unordered Map) storing the Key-Value pairs for the partitions it currently owns.

4.4.2 Consistency Enforcement (Synchronous Replication)

To satisfy the thesis requirement of "Strong Consistency" without the overhead of Raft-per-key, Workers implement a Synchronous Primary-Backup protocol.

When a Worker receives a SET request, it executes the following atomic flow:

1. **Guard Check:** The Worker verifies against its local Partition Table that it is indeed the **Primary** for the requested key. If not, it returns WRONG_NODE.

2. **Lock & Local Write:** It locks the key and writes the value to local memory.

3. **Replication:** It identifies the **Backup Worker** for this partition and sends a REPLICATE command.

4. **Barrier:** The Worker **blocks** and waits. It does not send a response to the client.

5. **Acknowledgement:** Once the Backup Worker confirms the write (ACK), the Primary unlocks the key and returns OK to the client.

This architecture ensures that any write acknowledged to the client exists physically on at least two servers, providing durability against single-node failures.

4.5 Data partitioning

The design of any distributed system at it's conception solve the problem of partinioning the data. As the application can grow infinitely, physical limits are set. Our computer have limited capacity for memory, CPU and network I/O, problem known by vertical scaling. The solution is horizontal scaling, which is **distributing** the work on many machines and "sharding" the data.

This however introduces new challanges that are complex. We must answer some questions in design of our application:

- How will the data be divided? We must have a function f, that will map a key k to a specific node n from the set of N nodes in the cluster.
- How does client determine on which node is his data?
- What does the system do when a new node is added? What does it do when the node is removed?

4.5.1 Karger's Consistent Hashing

Proposed by Karger et al. at MIT in 1997, consistent hashing solves the "reshuffling" problem inherent in modular hashing. In a standard modulo setup ($\text{hash}(k) \pmod n$), changing n (adding/removing a node) results in nearly all keys being remapped to different nodes. Karger's approach maps both the **Nodes** and the **Keys** onto the same identifier space, typically treated as a circle or "ring" (e.g., integers 0 to $2^{32} - 1$).

- **Placement:** A node is assigned a position on the ring by hashing its ID (e.g., IP address). A key is assigned a position by hashing the key itself.
- **Lookup:** To find the node responsible for a key, we move clockwise around the ring from the key's position until we encounter a node. That node is the "coordinator" or owner of the key.
- **Scaling:** When a new node is added to the ring, it only "steals" keys from the node immediately following it in the clockwise direction. The rest of the keys in the cluster remain unaffected.

Mathematically, this ensures that when the n-th node is added, only $1/n$ of the keys need to be moved, which is the theoretical minimum amount of data movement required to maintain a balanced load.

Virtual Nodes: A pure implementation of Karger's ring often results in uneven data distribution due to the random placement of nodes. To solve this, modern implementations (like Cassandra and Dynamo) use "Virtual Nodes" (vnodes). A single physical machine acts as multiple nodes on the ring (e.g., 256 positions), ensuring a more uniform statistical distribution of keys.

4.5.2 Lamping And Veach's Jump Consistent Hash

In 2014, Google engineers Lamping and Veach introduced "Jump Consistent Hash," an algorithm that achieves the same stability properties as Karger's ring but with significantly higher performance and zero memory overhead.

Unlike Karger's approach, which requires storing the ring structure in memory (typically a binary search tree or sorted map) to look up a node, Jump Hash computes the bucket assignment algorithmically using a Pseudo-Random Number Generator (PRNG).

The logic is probability-based:

- When a cluster scales from n to $n+1$ buckets, the algorithm ensures that any specific key has exactly a $\frac{1}{n+1}$ probability of moving to the new bucket $n+1$.
- The algorithm uses the key to seed the PRNG and "jumps" forward through the sequence of bucket sizes to find the final destination.

While extremely fast (executing in nanoseconds), Jump Consistent Hash has a strict constraint: it only supports scaling at the tail. You cannot arbitrarily remove "Node 2" from a 5-node cluster; you can only scale down from 5 to 4. This makes it ideal for data storage systems where buckets are logical shards, but less flexible for direct mapping to dynamic physical servers.

4.5.3 The sorted monolithic map

This approach, also known as **Range Partitioning**, rejects the randomization of hashing entirely. Instead, it treats the entire key-space as a single, sorted monolithic map.

The data is ordered lexicographically (by byte comparison) and divided into contiguous ranges (often called "Tablets," "Regions," or "Ranges").

- **Routing:** The system maintains a global index (like a B-Tree) that maps ranges to nodes. For example, keys A, K might map to Node 1, while keys K, Z map to Node 2.

- **Range Scans:** Unlike hash partitioning, this model allows for efficient range queries (e.g., 'SCAN user:100:log:*'). Since the keys are physically stored together, the system can fetch them in a single disk seek or network call.

However, this design suffers from the **"Sequential Write" problem**. If an application generates keys with a monotonically increasing prefix (e.g., timestamps or auto-incrementing IDs), all write traffic will target the single node responsible for the end of the range, creating a massive hot spot while other nodes sit idle. This necessitates complex splitting and rebalancing logic, as seen in systems like TiKV and CockroachDB.

4.6 Analysis of standing systems

4.6.1 Redis cluster: Pre-sharded "Hash slots"

The key-space is pre-sharded into a fixed, static number of 16,384 "hash slots". This number is constant regardless of the number of nodes in the cluster. A key is mapped to a slot using the formula:

$$\text{hash\_slot} = \text{CRC16}(\text{key}) \pmod{16384}$$

. Each primary node in the cluster is responsible for a subset of these 16,384 slots. A 3-node cluster, for example, might be configured as:

- Node A: Hash slots 0 - 5500
- Node B: Hash slots 5501 - 11000
- Node C: Hash slots 11001 - 16383

Load balancing is an explicit, administrator-driven process called "resharding". An administrator uses the redis-cli tool to specify how many slots to move from a source node to a target node. This operation can be performed with "no downtime".

Request Routing (Client-Side Intelligence): This is the most complex component of the Redis Cluster design. Cluster nodes do not proxy requests. If a client sends a request to a node that does not own the key's hash slot, the node replies with a redirection error. There are two types of redirection:

- MOVED: This is a permanent redirection. It informs the client that the hash slot is now permanently owned by a different node. The client must update its local cluster map and retry the request at the new node.
- ASK: This is a temporary redirection used during a resharding operation. When a slot is being migrated, it is in a MIGRATING state on the source and

IMPORTING state on the target. A -ASK redirect tells the client, "The key might be on the new node, but only for this one request. Do not update your map.". The client must then send an ASKING command to the target node before sending the redirected query. This ASKING command authenticates the client to access a key in an IMPORTING slot.

Key Colocation: Redis provides a "hash tags" feature to manually force keys into the same slot. If a key contains a ... substring, only the content inside the brackets is hashed. For example, user:123:profile and user:123:account are guaranteed to be in the same hash slot, enabling multi-key operations on them.

Redis's "hash slot" model represents a pragmatic compromise. It sacrifices the automatic, self-healing nature of consistent hashing for simplicity, predictability, and explicit control. The cluster state is simply a 16,384-element array mapping slots to nodes. This is far simpler to manage, gossip, and reason about than the mathematics of a continuous ring. The trade-off is twofold: load balancing is a manual operation, and the server logic is simplified at the cost of massively complicating the client, which must implement the sophisticated -MOVED/-ASK redirection protocol.

4.6.2 Amazon Dynamo

Amazon's Dynamo, as documented in the 2007 SOSP paper, is the archetypal highly available, "always-on" key-value store.

Model: It uses the classic Karger-style consistent hashing, enhanced with virtual nodes to address load balancing and heterogeneity.

Replication & Partitioning: Dynamo's "preference list" is a key concept. For a given key k with a replication factor of N, its preference list is composed of the first N distinct physical nodes encountered by walking the ring clockwise from k's position. The first node on this list acts as the "coordinator" for writes. https://www.allthingsdistributed.com/files/am dynamo-sosp2007.pdf https://www.cs.cornell.edu/courses/cs6410/2025fa/slides/19-dynamo.pdf

Load Balancing: Balancing is inherent to the virtual node model. When a new node is added, it assumes responsibility for numerous small virtual ranges from all other nodes on the ring, thus automatically distributing the load. https://www.allthingsdistributed.com/files dynamo-sosp2007.pdf

Request Routing: "Any storage node" in Dynamo is eligible to receive a client request. That node then acts as the coordinator, enforcing a quorum-like technique (e.g., R+W¿N) among the nodes in the key's preference list. https://www.allthingsdistributed.com/files/ama dynamo-sosp2007.pdf

While Dynamo's partitioning model is foundational, its overall architecture is philosophically opposed to a Raft-based system. Dynamo is the canonical "AP"

(Available, Partition-Tolerant) system, designed to "sacrifice consistency under certain failure scenarios" to achieve its "always-on" goal. Its entire design—built on eventual consistency , object versioning with vector clocks , and application-assisted conflict resolution —is intended to manage and resolve data conflicts.

Raft, by contrast, is a consensus algorithm designed to build "CP" (Consistent, Partition-Tolerant) systems. Its entire purpose is to prevent conflicts and enforce strong, linearizable consistency. Therefore, while Dynamo's partitioning model is relevant, its overall goals and consistency mechanisms are a poor fit for a Raft-based project.

4.6.3 Hazelcast IMDG

Hazelcast is an in-memory data grid (IMDG) that, like Redis, uses a hash-partitioned model.

Model: The key-space is divided into a fixed, configurable number of partitions. The default is 271.

Hashing Algorithm: Hazelcast uses MurmurHash3, a high-quality, non-cryptographic hash function. The mapping is $partition_id = MurmurHash3(key_bytes) mod 271$. The default of 271 is chosen because it is a prime number, which helps to further ensure an even distribution of hash results.

Load Balancing: This is the most subtle and insightful aspect of Hazelcast's design. The documentation states: "Thanks to the consistent hashing algorithm, only the minimum amount of partitions are moved" when a member joins or leaves. This appears contradictory; a modulo-based system is precisely what consistent hashing was designed to replace.

This contradiction is resolved by a "Two-Level Hashing" hybrid architecture. Hazelcast uses two different hashing mechanisms for two different purposes:

Level 1 (Key -¿ Partition): This mapping is static, using MurmurHash3(key)

Level 2 (Partition -¿ Member): This mapping is dynamic. A consistent hashing algorithm (or a similar structure) is used to map the 271 partitions (e.g., p-0 through p-270) onto the N available physical cluster members.

This two-level design provides the best of both worlds. When a new member joins, the Level 1 mapping is unchanged. The Level 2 partition-to-member consistent hash map is updated, determining that a minimal set of partitions (not keys) should be migrated to the new member. This architecture achieves the simplicity and fine-grained "virtual" units of Redis's hash slots, but adds the automatic, self-healing load balancing of Dynamo's consistent hashing.

Raft Integration (CP Subsystem): This provides a direct, practical blueprint for a Raft-based cache. By default, Hazelcast IMDG is an "AP" system, using "lazy

replication" and "best-effort consistency". However, it includes an optional "CP Subsystem" that is Raft-based.

When the CP Subsystem is disabled ("unsafe mode"), CP data structures (like IAtomicLong or FencedLock) "operate in unsafe mode" and use the standard, fast, lazy-replicated partitioning. In this mode, a write "can be lost" on a crash. https://docs.hazelcast.com/hazelcast/5.6/faq

When the CP Subsystem is enabled, these specific structures are linearizable, "prefer consistency over availability," and are managed by the Raft consensus mechanism.

This "bifurcated consensus model" is a critical architectural pattern. It separates the system into: 1) a high-speed, AP, partitioned data plane for the cache data itself, and 2) a separate, high-consistency, CP (Raft) control plane for coordination and metadata (e.g., locks, leader election, semaphores). This strongly suggests that using Raft for every SET operation in a cache is likely the wrong design, but using Raft to manage the cluster's state is an excellent one.

4.6.4 TikV

TiKV is a distributed, transactional key-value store that serves as the storage layer for the TiDB database.

Model: Data is partitioned by key range.

Partitioning Unit: The basic unit of data movement and consensus is called a "Region". A Region represents a contiguous range of keys in the sorted map.

Consensus Integration: This is the core of the design. Each Region is an independent Raft group. Each Region has multiple replicas (called "Peers"), typically 3, one of which is the Raft leader that services all reads and writes for that key-range.

Load Balancing: Balancing is automatic, dynamic, and centralized, managed by the "Placement Driver (PD)".

The PD is the "brain" of the cluster , storing the metadata for all Regions.

It balances storage size by instructing Regions to split when they grow too large (e.g., ¿ 96-144MB) and to merge with adjacent Regions when they become too small.

It balances load by automatically migrating Regions (Raft groups) between different nodes.

It balances hot spots by actively transferring the Raft leadership of a hot Region to a peer on a less-loaded node.

Request Routing: A client asks the PD to identify which Region (and its leader) is responsible for a given key.

4.6.5 Cockroach Db

CockroachDB is a distributed SQL database that employs an architecture very similar to TiKV's.

Model: It uses a "monolithic sorted map" of key-value pairs.

Partitioning Unit: The partitioning unit is called a "Range" (conceptually identical to TiKV's Region).

Consensus Integration: Identical to TiKV. The "replication layer" ensures that each Range is a Raft group.

Load Balancing: Automatic and size-driven. Ranges automatically split when they grow too large and merge when they become too small. The cluster automatically rebalances these ranges across all available nodes, for example, when a new node joins or an old one fails.

Request Routing (Decentralized Index): This is CockroachDB's primary architectural divergence from TiKV. It does not have a centralized PD for routing. Instead, it embeds the routing information within the key-space itself in a special index called "meta ranges".

This index is a two-level tree: the root (meta1) points to second-level ranges (meta2), and the meta2 ranges point to the "user data" ranges.

To find a key, a node performs a "bottom-up" lookup in this tree (which is itself composed of standard, Raft-replicated ranges), caching the results for future use.

This design represents a clear trade-off in metadata management. TiKV uses an external, centralized (though highly-available) component, the PD, which simplifies routing logic. CockroachDB uses a decentralized, self-referential model where the metadata is "just data" stored in special ranges.

4.6.6 Google Spanner

Google's Spanner is the conceptual and technical ancestor to systems like TiKV and CockroachDB. It is a globally-distributed, synchronously-replicated database.

Model: Spanner partitions data into "tablets" (similar to Bigtable) and uses "directories" as the unit of data placement and migration.

Consensus Integration: It "shards data across many sets of Paxos state machines". (Raft is a modern, more understandable successor to the Paxos consensus algorithm; the architectural model is the same).

Load Balancing: The system is automated and managed by a hierarchy of components: a "placement driver" handles data movement across zones, while "zonemasters" assign data to "spanservers".

Request Routing: Clients use "per-zone location proxies" to find the correct spanserver serving their data.

The fact that Google (Spanner) , TiKV , and CockroachDB all independently converged on the same fundamental architecture—a sorted key-space, partitioned into ranges, with each range managed by its own consensus group (Paxos/Raft)—is one of the most powerful trends in modern distributed systems. TiKV's design, for instance, is explicitly "based on the design of Google Spanner". This "Raft-per-Shard" (or "Paxos-per-Shard") model is the state-of-the-art for building strongly-consistent, scalable databases.

4.7 Node

Node is a set of Raft servers running and reaching a consensus at the same time at all times.

4.8 Server

Server is a single machine running a Raft algorithm.

4.9 Application Database

Application Database is an actual cache that we keep - this is where all the logic around saving, getting and deleting data is being done. We can access this layer if, and only if, the Raft node reached a consensus and majority of the servers have agreed upon it.

Chapter 5

Implementation

1. Why many threads, how synchronization works between them

Chapter 6

Testing

Chapter 7

Deployment

1. Dockery 2. Use in other languages - API, installation 3. Kubernetes?

Chapter 8

Benchmarks

Chapter 9

Summary and ?Future work?